

一. 解题思路:

1. 存储题目，然后计算数字黑格周围的白格数。

2. 首先在 0 周围标记不能放灯，在 4 周围放灯并照亮上下左右。

3. 然后进行简单求解

- 若周围白格数与黑格数字相同或需求减周围的灯数等于白格数，则在黑格旁的白格放灯。
- 若某个白格，只能被唯一地方的灯照亮，则在此处放灯。
- 若发现违反规则
(黑格周围灯数大于需求、灯数加可放灯的白格数小于需求，有的白格没有地方可以放灯照亮它，灯泡照到其他灯泡) 则返回 false。
- 循环进行，直至图表不在改变，检查一遍图表的合理性 (不违反条件) 返回 true，若此时已解决则直接输出

4. 进行回溯

- 每次递归前进行简单求解
- 检查此时图表是否已经解完 (数字黑格满足需求且所有白格都被照亮)，若解完则输出。
- 若此时没有解完，则选择一个可能性最大的白格
(优先选择周围数字黑格更多的白格，如果没有找到周围有黑格的白格则遍历白格，计算可以照亮这个白格的位置数量，选择位置数量最少的白格，返回 true)
如果未解完且找不到可以放灯的白格则返回 false
- 将当前表格存下来
- 尝试放灯，如果成功，进行下一次递归
- 否则恢复原来的图表
- 尝试禁止放灯，如果成功，进行下一次递归
- 直到成功返回 true，若失败则返回 false

二. 重点函数:

```
void copychartBtoA(grid** A, grid** B); //复制图表
```

```
void input(grid** chart, string s); //输入图表,并将数字黑格存入列表,并求黑
```

C++

格周围白格数

```
void printgrid(grid** N); //打印图表
bool isblack(grid** c, int x, int y); //检查是否为数字黑格
bool isin(int x, int y); //检查该位置是否在图表范围内
void countwhiteandlight(grid** c); //计算黑格周围白格与灯数
bool isvalid(grid** c); //检查图表是否合理
bool canlight(grid** c, int x, int y); //检查是否可以放置灯
bool lightup(grid** c, int x, int y); //点亮该位置灯泡
bool isfinish(grid** c); //查看是否已经解决
void handle0and4(grid** chart); //在开始前处理0,4黑格
bool notlight(grid** c, int x, int y); //设置不能放灯
void candidate(grid** c, int x, int y, VecList<black>* can, int* n); //
搜索可以放灯点亮此格的位置
bool simplehandle(grid** c); //处理一些简单能确定的灯泡，并更新黑格
bool chooseWhiteCell(grid** c, int& wx, int& wy); //选择可能性最大的白格
bool hardhandle(grid** c, int depth); //回溯求解
```

三. 数据结构:

向量表

```
//每格坐标及其属性
//空格为-2, 黑格为-1, 0-4的黑格分别为0-4, 5为灯, 点亮为6, 禁止放灯为-3
typedef struct
{
    int type; //属性
    int aroundempty; //周围空格数
    int aroundlight; //周围灯数
}grid;

//存储黑格或候选照亮点坐标
typedef struct
{
    int x;
    int y;
}black;
```